WUMPUS 3D · RELATÓRIO DE REFATORAÇÃO

Código Limpo

O que mudou na organização do código, como foi comprovado que o jogo continua igual e o que se ganhou em leitura, manutenção, memória e repetição.

Base	**commit 8544609 (main)**
Branch	**refatoracao**
Data	**3 de novembro de 2026**
Referência	**MARTIN, Robert C. *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River: Prentice Hall, 2008. ISBN 978-0-13-235088-4. (Em português: *Código Limpo*, Alta Books.)**

Sumário

1 Resumo

2 Princípios aplicados

3 O que mudou, parte por parte

4 Como foi comprovado que nada quebrou

5 Benefícios

6 Prévia do jogo depois da refatoração

7 O que ficou para depois

1 Resumo

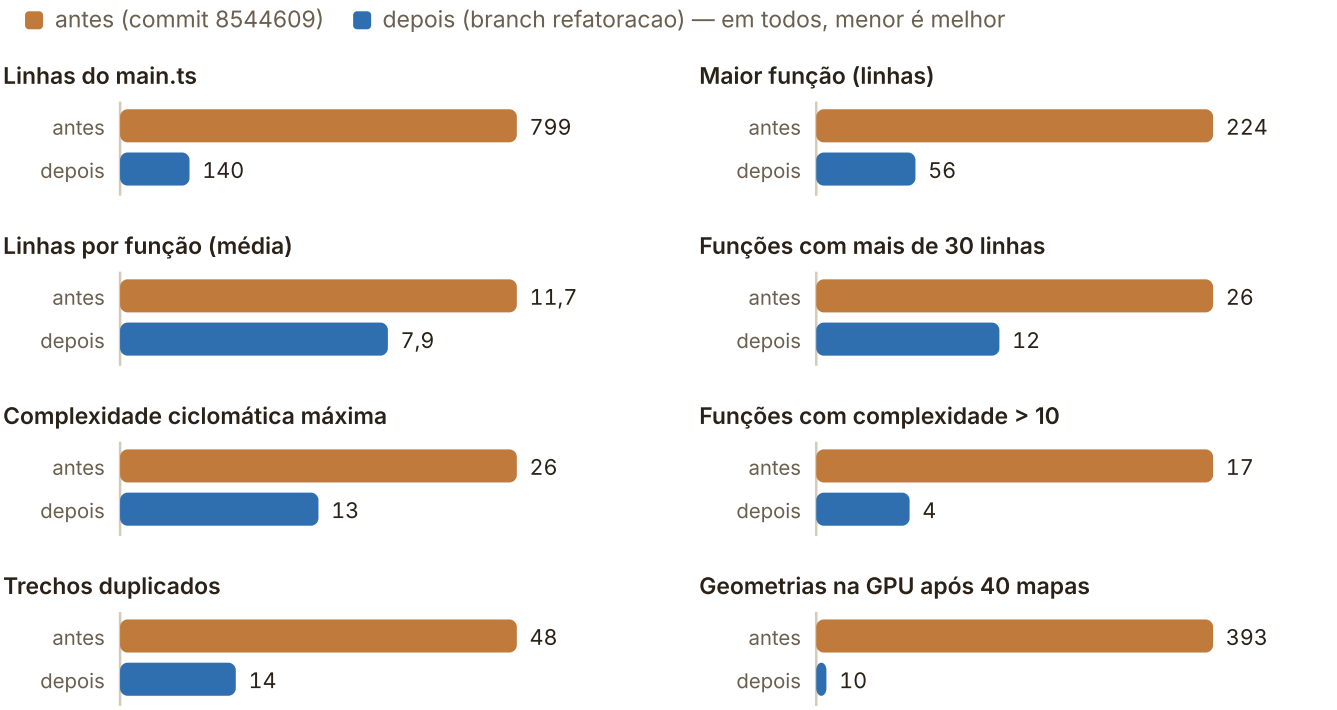
O código do Wumpus 3D foi reorganizado seguindo os princípios do livro *Código Limpo*, de Robert C. Martin: funções pequenas que fazem uma coisa só, nomes que explicam a intenção, nada de repetição, e cada módulo com uma única responsabilidade. **Nenhuma regra do jogo, pontuação, sorteio de mapa ou decisão de agente mudou** — e isso foi comprovado, não só suposto (capítulo 4).

−82%
linhas no main.ts (799 → 140)

−75%
tamanho da maior função (224 → 56 linhas)

−71%
trechos duplicados (48 → 14)

−97%
geometrias acumuladas na GPU após 40 mapas (393 → 10)



O QUE FOI FEITO, EM UMA FRASE POR PARTE

- **Interface**: o `main.ts`, que fazia tudo, virou o ponto de entrada de 9 módulos em `src/ui/` (um por seção do painel, mais estado, HUD e controle da partida).
- **Regras**: `step()` (102 linhas) virou um método por ação; o sorteio do mapa saiu do construtor.
- **Agentes**: as decisões longas (`explore()`, 40 e 64 linhas) foram divididas em passos com nome.
- **3D**: `load()`, `sync()` e `frame()` divididos; a cena agora libera a memória de vídeo do mapa anterior.
- **Arena, relatórios, armazenamento e scripts**: funções gigantes divididas e utilitários repetidos reunidos num lugar só.

2 Princípios aplicados

Cada mudança segue uma regra do livro. A tabela liga o princípio ao que mudou no projeto:

PRINCÍPIO (CÓDIGO LIMPO)	ANTES	DEPOIS
Funções pequenas (cap. 3)	`initArena` com 224 linhas; `step` com 102; `load` com 92.	Maior função com 56 linhas (um gráfico SVG); média de 7,9 linhas.
Fazer uma coisa só (cap. 3)	`runCompetition` lia a tela, jogava, montava o ranking e o relatório.	`readSettings`, `playMaps`, `summarize`, `runCompetition` só coordena.
Um nível de abstração por função (cap. 3)	`explore()` misturava busca de caminho, probabilidade e textos.	`explore` chama `planShot`, `gamble`, `riskMeasure`, `evidenceCount`.
Não se repita (DRY) (cap. 17)	`$` definido 3 vezes; `esc` 2; `pct/num` 7; laço de partida offline 3; download, cópia e confirmação em 2–3 lugares.	`src/ui/dom.ts`, `src/util/format.ts`, `playToEnd` / `playMaps`, `rankingCells`, `confirmTwice`.
Nomes que revelam a intenção (cap. 2)	`fmt`, `pct`, `g`, `r`, `k.split(',').map(Number)` espalhado.	`cellName`, `percent`, `parseKey`, `firingSpots`, `followPlan`, `prefersShooting`.
Responsabilidade única (cap. 10)	`main.ts`: estado, HUD, mapas, treino, avaliação, partidas humanas e eventos.	Um módulo por responsabilidade em `src/ui/`; `main.ts` só liga as peças.
Código morto sai (cap. 17)	`benchmark()` sem uso; `fmt` / `pct` exportados sem uso.	Removidos.
Tabela em vez de if/else (cap. 3, switch)	`parseArgs` com cadeias de `else if` nos scripts; teclas e botões em sequências de `if`.	`parseFlags` (`scripts/args.ts`), mapas `COMMON_KEYS` e `clicks` em `main.ts`.
Comentários explicam o porquê (cap. 4)	Bons comentários já existiam.	Mantidos; cada função nova tem uma linha dizendo para que serve.

> **O QUE A REFATORAÇÃO NÃO MUDOU DE PROPÓSITO**
>
> - Nenhuma das 8 invariantes do projeto (`CLAUDE.md`) e nenhuma decisão D1–D20.
> - Sem framework de interface (D1 continua valendo): os módulos são funções simples.
> - O id dos agentes importados continua com a mesma fórmula, para não perder agentes já salvos nos navegadores.
> - Os 41 testes existentes não foram alterados.

3 O que mudou, parte por parte

Interface: o main.ts deixou de fazer tudo

Antes, 799 linhas misturavam estado da partida, placares, mapas, treino genético, avaliação, partidas humanas e eventos. Mexer em "Salvar mapas" exigia ler o arquivo inteiro. Agora cada seção é um módulo, e o `main.ts` ficou assim (o arquivo inteiro tem 140 linhas):

src/main.ts — o fim do arquivo: cada linha liga uma parte

```
window.addEventListener('keydown', onKeyDown);
initGameButtons();
initConfigFields();
initMapsPanel();
initGeneticPanel();
initBenchPanel();
initHumanPanel(() => restart());
initAnalysis();
initArenaScreen();

setSetupOpen(window.innerWidth > 760);
refreshPlayerOptions();
updateSetup();
updatePitHint();
updateHumanPanel();
restart();
view.renderer.setAnimationLoop(tick);
```

Repetição: escrito uma vez, usado em vários lugares

antes — a mesma lógica em 3 arquivos

```
// main.ts — excluir mapa
if (!delBtn.dataset.confirm) {
  delBtn.dataset.confirm = '1';
  delBtn.textContent = 'Confirmar?';
  return;
}
// arena/ui.ts — remover agente
if (!del.dataset.confirm) {
  del.dataset.confirm = '1';
  del.textContent = 'Confirmar?';
  return;
}
// main.ts — apagar partidas
if (!btn.dataset.confirm) {
  btn.dataset.confirm = '1';
  btn.textContent = `Confirmar: apagar ...`;
  return;
}
```

depois — confirmTwice() em src/ui/dom.ts

```
// src/ui/dom.ts — uma vez só
export function confirmTwice(btn, opts) { ... }

// maps-panel.ts
confirmTwice(deleteButton, {
  canStart: () => !!selectedMap(),
  confirmLabel: () => 'Confirmar?',
  onConfirm: () => { deleteMap(...); ... },
});
// arena/ui.ts
confirmTwice(del, {
  confirmLabel: () => 'Confirmar?',
  onConfirm: () => removeAgent(external),
});
// human-panel.ts
confirmTwice(clear, {
  confirmLabel: () => `Confirmar: apagar ...`,
  onConfirm: () => { clearHumanGames(...); ... },
});
```

Regras: um método por ação

antes — world.ts, step() (8544609)

```
step(action: Action): StepResult {
  // ... cobra a ação ...
  switch (action) {
    case 'left': ...
    case 'right': ...
    case 'forward': {
      const [dx, dy] = DIR_VEC[a.dir];
      const nx = a.x + dx;
      const ny = a.y + dy;
      if (!this.inBounds(nx, ny)) {
        bump = true;
        message = 'Bateu na parede!';
        break;
      }
      a.x = nx; a.y = ny;
      this.visited.add(key(nx, ny));
      message = `Andou para (${nx + 1}, ${ny + 1}).`;
      if (this.hasPit(nx, ny)) {
        this.status = 'dead-pit';
        this.score += SCORE.death;
        message = 'Você caiu em um poço! ...';
      } else if (this.wumpusAlive && ...) {
        // ... mais 15 linhas ...
      }
      break;
    }
    case 'shoot': { /* 30 linhas */ }
    case 'grab': ...
  }
  // ... limite de ações, percepções ...
}  // 102 linhas no total
```

depois — step() e moveForward()

```
step(action: Action): StepResult {
  if (this.gameOver) return { ... };
  this.score += SCORE.action;
  this.actions++;
  const outcome = this.perform(action);
  if (this.status === 'playing' && this.actions >=
this.maxActions) {
    this.status = 'timeout';
    outcome.message = `Limite de ...`;
  }
  this.percepts = this.sense(outcome.bump,
outcome.scream);
  return { percepts: this.percepts, message:
outcome.message, arrow: outcome.arrow };
}

private moveForward(): Outcome {
  const a = this.agent;
  const [dx, dy] = DIR_VEC[a.dir];
  const next = { x: a.x + dx, y: a.y + dy };
  if (!this.inBounds(next.x, next.y)) return {
message: 'Bateu na parede!', bump: true };
  a.x = next.x;
  a.y = next.y;
  this.visited.add(key(next.x, next.y));
  if (this.hasPit(next.x, next.y)) return
this.die('dead-pit', '...');
  if (this.wumpusAt(next)) return this.die('dead-
wumpus', '...');
  if (next.x === 0 && next.y === 0) return
this.arriveAtEntrance();
  return { message: `Andou para (${next.x + 1},
${next.y + 1}).` };
}
```

Com um método por ação, a regra de morte, a de vitória (D6) e a do tiro ficam cada uma num lugar com nome. O sorteio do mapa também saiu do construtor para `randomPlacement()`, preservando a ordem exata das chamadas ao sorteador — por isso os mapas de cada semente continuam os mesmos (invariante 7).

Todos os arquivos tocados

ARQUIVO	LINHAS ANTES	LINHAS DEPOIS	O QUE MUDOU
`src/main.ts`	799	140	Só liga teclado, botões e painéis aos módulos.
`src/ui/state.ts`	—	60	novo: estado da partida (`match`) e campos de configuração.
`src/ui/game.ts`	—	185	novo: `act`, `agentStep`, `restart`, câmera, laço `tick`.
`src/ui/hud.ts`	—	110	novo: placares, percepções, pensamento, registro e fim de jogo.
`src/ui/players.ts`	—	50	novo: quem joga, seletor, criação do agente.
`src/ui/maps-panel.ts`	—	111	novo: seção "Salvar mapas".
`src/ui/genetic-panel.ts`	—	154	novo: seção "Agente genético".
`src/ui/bench-panel.ts`	—	77	novo: seção "Avaliação".
`src/ui/human-panel.ts`	—	125	novo: seção "Minhas partidas" e árbitro da partida humana.

ARQUIVO	LINHAS ANTES	LINHAS DEPOIS	O QUE MUDOU
`src/ui/dom.ts`	—	79	novo: `$`, mensagens, download, copiar, confirmar, telas sobrepostas.
`src/util/format.ts`	—	26	novo: porcentagens, números, escape de HTML, nomes de arquivo (sem DOM).
`src/game/world.ts`	357	382	`step` por ação; `randomPlacement`, `reachable`, `parseLayoutCell`; `parseKey`, `gridNeighbors`, `inGrid`, `sameCell`.
`src/agents/agents.ts`	337	404	Classe base `ThinkingAgent`; `explore` dividido; características do genético em `cellFeatures`.
`src/agents/knowledge.ts`	162	172	Enumeração de poços em `enumeratePits`; vizinhas pela grade comum.
`src/agents/genetic.ts`	127	142	`validate`, `tournament`, `nextGeneration` saíram de `train`.
`src/agents/benchmark.ts`	39	27	`playToEnd` / `playMaps` (usados por treino, medir e treinar); `benchmark()` morto removido.
`src/render/view.ts`	421	485	`load` em 5 construtores; `sync` e `frame` divididos; `disposeTree`.
`src/arena/ui.ts`	330	353	`initArena` (224 linhas) em 15 funções; tabela usa `rankingCells`.
`src/arena/ranking.ts`	82	95	`rankingCells` compartilhado com a tela.
`src/arena/runner.ts`	83	99	`failedEpisode` (antes montado à mão no script da Arena).
`src/arena/protocol.ts`	102	106	`normalizeBeliefs` separado.
`src/report/ui.ts`	125	118	`render` dividido; usa `dom.ts`.
`src/storage.ts`	153	125	`readJson` / `readList` / `writeJson` / `upsert` no lugar de 13 blocos try/catch.
`scripts/arena.ts`	151	183	`main` (66 linhas) em `loadParticipants`, `playAll`, `writeReport`.
`scripts/args.ts`	—	15	novo: opções de linha de comando como tabela (arena e treinar).

Também mudaram, sem alterar o conteúdo dos textos: `report/html.ts` e `report/charts.ts` (usam `format.ts`), `report/report.ts` (`slugify`), `scripts/medir.ts` e `scripts/treinar.ts` (usam `playMaps`).

4 Como foi comprovado que nada quebrou

Refatorar é mudar a estrutura **sem mudar o comportamento**. Antes de mexer em qualquer linha, a versão original gravou uma "linha de base"; depois, as mesmas execuções foram repetidas e comparadas.

VERIFICAÇÃO	COMO	RESULTADO
Tipos e testes	`npm run check` (tsc + Vitest)	**49 de 49** passando: os 41 originais, sem alteração, e 8 novos (`tests/refatoracao.test.ts`).
Arena 4×4	300 mapas, semente `verificacao`, JSON completo e relatório com as 1500 partidas	**Idêntico** (fora tempo de decisão e data).
Arena 6×6	150 mapas, semente `verificacao6`	**Idêntico**.
Medição	`npm run medir` com 2000 mapas 4×4 e 500 6×6	**Idêntico**, incluindo os intervalos de confiança.
Treino genético	`npm run treinar` com semente fixa	**Mesmo genoma**, mesmas aptidões por geração.
Ranking em Markdown	Arena só com o agente de exemplo	**Idêntico**.
Navegador	Mesmo roteiro do Playwright nas duas versões, com `Math.random` semeado: teclado, 4 agentes passo a passo e jogando sozinhos, avaliação, salvar/carregar/exportar/importar/excluir mapas, desafio por semente, Minhas partidas, Análise, Arena com agente importado (assistir e remover), treino genético, celular	**67 de 67** pontos de verificação idênticos; nenhum erro no console; sem rolagem horizontal no celular (390 px).
Memória de vídeo	40 mapas novos seguidos (tecla R), lendo `renderer.info.memory`	Antes: 9 → **393** geometrias. Depois: 10 → **10**.

POR QUE O AGENTE ALEATÓRIO FICOU DE FORA DAS COMPARAÇÕES POR SEMENTE

Ele usa `Math.random`, que não tem semente nos scripts: dá números diferentes a cada execução, antes e depois da refatoração. Todos os outros agentes são determinísticos e deram resultados idênticos.

UM DETALHE DO TESTE NO NAVEGADOR

Com `Math.random` semeado, a primeira comparação divergiu no "novo mapa": o Three.js também usa `Math.random` para criar o identificador de cada geometria e material, e a versão nova cria **menos** objetos (os poços passaram a compartilhar geometria). Uma build de teste com os poços criados como antes deu os 67 pontos idênticos, isolando a causa. Em uso normal, sem semente, isso não tem efeito visível.

5 Benefícios

Manutenção e leitura

- **Achar onde mexer ficou direto.** "Salvar mapas" está em `maps-panel.ts`; a regra de vitória, em `arriveAtEntrance()`; a nota do tiro do genético, em `prefersShooting()`. O nome do arquivo ou da função diz o que tem dentro.
- **Menos lugares para corrigir o mesmo erro.** Download, cópia, confirmação, formatação de porcentagem e a partida sem 3D agora têm um só código. Antes, uma correção no download precisava ser feita em dois arquivos.
- **Funções testáveis.** Peças que só existiam dentro de funções grandes viraram funções com entrada e saída (`enumeratePits`, `playMaps`, `rankingCells`, `slugify`, `gridNeighbors`) e ganharam testes.
- **Complexidade menor.** Complexidade ciclomática média 2,96 → 2,22; funções com 3 ou mais níveis de aninhamento 12 → 7. Menos caminhos por função = menos casos para pensar ao mudar.

Otimização

- **Vazamento de memória de vídeo corrigido** (estava nas pendências do HISTÓRICO): `disposeTree()` libera geometrias, materiais e texturas ao trocar de mapa, ao redesenhar as crenças do agente e quando a flecha termina. Em sessões longas, a memória da placa de vídeo para de crescer.
- **Poços compartilham geometria e material**: um mapa 6×6 com 7 poços cria 4 objetos em vez de 28.
- **Velocidade dos agentes**: a lógica de decisão é a mesma, passo a passo; não houve ganho nem perda de desempenho a medir.

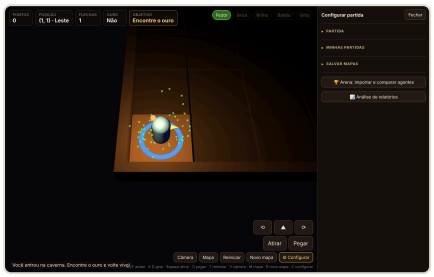
Redução de código

> **SENDO HONESTO SOBRE O TOTAL DE LINHAS**
>
> O total **aumentou** 12%. Dividir uma função de 100 linhas em seis funções pequenas acrescenta seis assinaturas, seis linhas de comentário dizendo para que cada uma serve e linhas em branco entre elas. O *Código Limpo* aceita essa troca: o objetivo é que cada pedaço seja fácil de entender, não que o arquivo seja o mais curto possível. A redução real está onde importa — no arquivo que fazia tudo (−82%) e nas repetições (−71%). O bundle cresceu 2,6% (gzip) pelos nomes de funções a mais; ele é dominado pelo Three.js (~600 KB), uma pendência antiga.

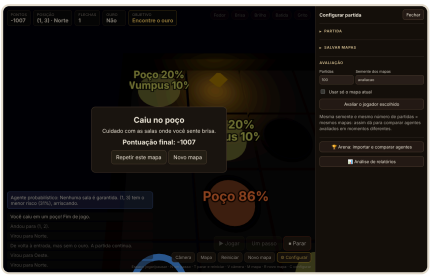
MEDIDA	ANTES	DEPOIS	VARIAÇÃO
`main.ts`	799	140	−82%
Trechos duplicados (ocorrências)	48	14	−71% (o que sobra é a definição única de cada um)
Código morto	`benchmark()`, 2 exportações sem uso	nenhum	removido
`storage.ts`	153	125	−18%
`agents/benchmark.ts`	39	27	−31%
Linhas totais em `src/` e `scripts/`	4324	4862	+12%
Linhas de código, sem comentários e linhas vazias	3605	3902	+8%

6 Prévia do jogo depois da refatoração

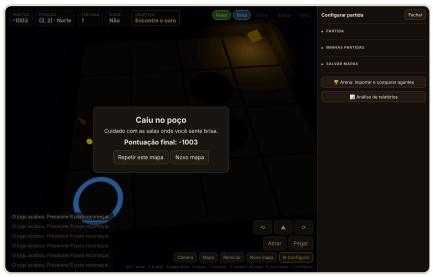
Capturas feitas pelo roteiro de verificação, na build de produção da branch `refatoracao` (computador 1280×800 e celular 390×844). A aparência é a mesma da versão anterior.



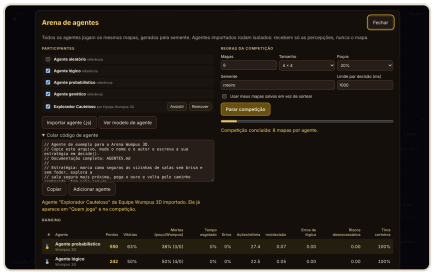
Início de partida, com o painel aberto.



Agente probabilístico jogando: crenças sobre as salas desenhadas no piso e o pensamento do agente.



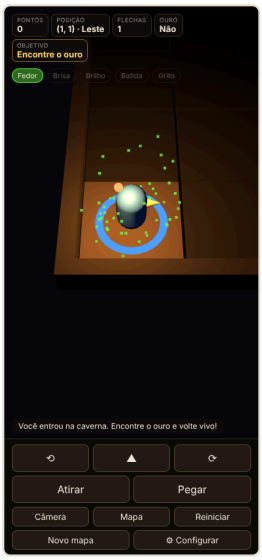
Mapa aberto (tecla M).



Arena: competição com o agente de exemplo importado.



Análise de relatórios: você × agentes nos mesmos mapas.



No celular.

7 O que ficou para depois

- Funções ainda acima de 30 linhas: `installChartHover` (56), `reportSections` (42), `renderReportHtml` (40) e o gráfico do treino (35). São montagens de HTML/SVG com pouca lógica (complexidade 2 a 5); dividi-las mais espalharia o desenho sem ganho de clareza.
- Complexidade ainda acima de 10: `parseLayout` e `parseReport` (validações em sequência, uma por campo), `Referee.judge` e `agentStep` (11). Cada `if` ali é uma regra a conferir.
- Testes automáticos de interface: o roteiro do Playwright usado aqui comparou as duas versões, mas não está no `npm run check`. É a pendência "testes de interface" do HISTÓRICO.
- Bundle de ~650 KB por causa do `import * as THREE` (pendência antiga).

Como as métricas foram medidas: análise da árvore sintática do TypeScript (compilador 5.6) sobre `src/` e `scripts/`, contando funções e métodos nomeados; complexidade ciclomática = 1 + `if`, `?:`, laços, `case`, `catch`, `&&`, `||` e `??`. Duplicações: contagem de padrões conhecidos (definições de `$`, `esc`, `pct`/`num`, download, cópia, confirmação, laço de partida offline, `split(',').map(Number)` e acessos ao `localStorage`).